
Contents

Helvetia System Manual	2
1. Installation Requirements	2
1.1 Required Software	2
1.2 Installation Instructions	3
1.3 System Requirements	5
1.4 Docker Resource Allocation	5
2. Building the System	6
2.1 Clone the Repository	6
2.2 Build the Java Application Server	6
2.3 Build the VTAdmin Web Interface	6
2.4 Build the Client Web Application	7
2.5 Set Up Python Environment	7
3. Running the System (Minimal Configuration)	7
3.1 Start the Vitess Cluster (DC1)	8
3.2 Start HDFS	8
3.3 Start Redis	8
3.4 Start the Java Application Servers	8
3.5 Start the Client Web Application	9
3.6 Access Grafana Dashboards	9
3.7 Restart VTAdmin Web (if needed)	9
4. Populating Data	9
4.1 Load Article Files into HDFS	9
4.2 Insert Users, Articles, and Reads into Vitess	10
4.3 Verify Data Population	10
5. Running Backup Tablets (Optional)	10
5.1 Create Backups Before Starting (Recommended)	11
5.2 Start Backup Tablets	11
5.3 Access Backup Replicas via Client	11
5.4 Test Failover	11
5.5 Stop Backup Tablets	12
5.6 Recover from Failed Startup	12
6. Running a Secondary Datacenter DC2 (Optional)	12
6.1 Start DC2	12
6.2 Validate in VTAdmin	13
6.3 Connect Client to DC2	13
6.4 Verify Independence	13

Helvetia System Manual

1. Installation Requirements

1.1 Required Software

Software	Minimum Version	Notes
Bazelisk	Latest	Build system orchestrator; automatically manages Bazel versions
Java (JDK)	17	Required for building and running the application server
Node.js	22.13.0	JavaScript runtime for web clients
npm	10.9.2	Node package manager (bundled with Node.js 22)
Docker	Latest	Container runtime for database clusters
Docker Compose	V2	Multi-container orchestration (included with Docker Desktop)
uv	Latest	Python package and environment manager
Python	3.13	Required for data population tools
MySQL client	Any	CLI client for database access
Git	Any	Version control

curl

Any

HTTP client for API
testing and health
checks

1.2 Installation Instructions

```
# Install Homebrew if not already installed
/bin/bash -c "$(curl -fsSL
↪ https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install Bazelisk
brew install bazelisk

# Install Java 17
brew install openjdk@17
# Add to PATH (add to ~/.zshrc for persistence)
export PATH="/opt/homebrew/opt/openjdk@17/bin:$PATH"
export JAVA_HOME="/opt/homebrew/opt/openjdk@17"

# Install Node.js 22
brew install node@22
# Add to PATH if needed
export PATH="/opt/homebrew/opt/node@22/bin:$PATH"

# Install Docker Desktop
brew install --cask docker
# Launch Docker Desktop and complete setup

# Install uv (Python package manager)
brew install uv

# Install Python 3.13
brew install python@3.13

# Install MySQL client
brew install mysql-client
export PATH="/opt/homebrew/opt/mysql-client/bin:$PATH"

# Verify installations
bazel --version
java -version
node --version
npm --version
docker --version
docker compose version
uv --version
```

```
python3.13 --version
mysql --version
```

macOS (using Homebrew)

```
# Update package lists
sudo apt update

# Install Bazelisk
sudo wget -O /usr/local/bin/bazel
  ↪ https://github.com/bazelbuild/bazelisk/releases/latest/download/bazelisk-
  ↪ linux-amd64
sudo chmod +x /usr/local/bin/bazel

# Install Java 17
sudo apt install openjdk-17-jdk
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64

# Install Node.js 22
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

# Install Docker
sudo apt install docker.io docker-compose-plugin
sudo usermod -aG docker $USER
# Log out and back in for group changes to take effect

# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# Install Python 3.13
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt install python3.13

# Install MySQL client
sudo apt install mysql-client

# Verify installations
bazel --version
java -version
node --version
npm --version
docker --version
docker compose version
uv --version
python3.13 --version
```

```
mysql --version
```

Linux (Ubuntu/Debian)

1.3 System Requirements

Resource	Minimum	Recommended	Full Deployment
RAM	16 GB	24 GB	32 GB
Disk Space	20 GB	50 GB	100 GB
CPU Cores	4	8	12

Note for Apple Silicon (M1/M2/M3) Macs: The Vitess Docker images use platform: linux/amd64, which requires x86_64 emulation via Rosetta. This is handled automatically by Docker Desktop but may result in slightly reduced performance.

1.4 Docker Resource Allocation

The system consists of multiple containerized components with the following memory requirements:

Component	Memory Usage	Containers	Description
DC1 (Primary)	~8 GiB	21	Primary datacenter with Vitess cluster
DC2 (Secondary)	~7 GiB	18	Secondary datacenter for geo-distribution
Backup Tablets	~1 GiB (idle) / ~6 GiB (active)	11	Backup replicas for DC1
HDFS	~1.6 GiB	3	Hadoop filesystem for article storage
Redis	~5 MiB	1	Caching layer
Monitoring	~1.1 GiB	4	Prometheus, Grafana, cAdvisor, Alertmanager

Deployment Configurations

Configuration	Components	Docker Memory	Use Case
Minimum	DC1 + HDFS + Redis	12 GB	Development/testing
Standard	DC1 + DC2 + HDFS + Redis	20 GB	Dual-datacenter testing
Full	Standard + Monitoring	22 GB	Full observability
Maximum	Full + Active Backup Tablets	28 GB	Complete HA setup

Docker Desktop Settings (Settings → Resources) For **Standard** deployment (recommended for evaluation): - **Memory:** 20 GB minimum (24 GB recommended) - **CPUs:** 6 cores minimum (8 recommended) - **Disk:** 50 GB - **Swap:** 2 GB

For **Minimum** deployment (development only): - **Memory:** 12 GB - **CPUs:** 4 cores - **Disk:** 30 GB

2. Building the System

2.1 Clone the Repository

```
git clone https://github.com/pwochnowski/helvetia.git
cd helvetia
```

2.2 Build the Java Application Server

The application server is built using Bazel:

```
# Build the server (first build downloads dependencies, may take several
↪ minutes)
bazel build //app:server

# Verify the build succeeded
bazel build //app:server_deploy.jar
```

2.3 Build the VTAdmin Web Interface

The VTAdmin web interface must be built locally before the container can serve it:

```
cd docker/vitess/web/vtadmin

# Install dependencies
npm install

# Build the production bundle
npm run build

# Return to project root
cd ../../../../..
```

Note: The Docker container serves the pre-built static files from `docker/vitess/web/vtadmin/build/`. If you modify the VTAdmin source, rebuild and restart the container.

2.4 Build the Client Web Application

```
cd client

# Install dependencies
npm install

# Return to project root
cd ..
```

2.5 Set Up Python Environment

```
# Create virtual environment using uv
uv venv activate

# Activate the environment
source activate/bin/activate

# Install dependencies
uv pip install -e .
```

3. Running the System (Minimal Configuration)

This section describes how to start the system with DC1 only (no data populated).

3.1 Start the Vitess Cluster (DC1)

```
cd docker/vitess

# Start DC1 primary tablets (schema loads automatically)
docker compose up -d

# Monitor startup progress (wait 2-3 minutes for tablets to initialize)
docker compose logs -f vtctld
```

Verify tablets are registered:

```
docker exec vitess-vtctld-1 vtctldclient --server localhost:15999 GetTablets
```

3.2 Start HDFS

```
cd ../hdfs

# Start HDFS cluster
docker compose up -d

# Verify HDFS is healthy (check http://localhost:9870)
docker exec hdfs-namenode hdfs dfsadmin -report

cd ../vitess
```

3.3 Start Redis

Redis is included in the Vitess docker-compose and starts automatically. Verify:

```
docker exec redis redis-cli ping
# Should return: PONG
```

3.4 Start the Java Application Servers

Start servers for both cells:

```
./app/run-primary-servers.sh
```

This starts: - **Server 1** on <http://localhost:8081> → VTGate Cell1 (Beijing) - **Server 2** on <http://localhost:8082> → VTGate Cell2 (HongKong)

3.5 Start the Client Web Application

In a new terminal:

```
cd client
npm run dev
```

The client is available at <http://localhost:3000>

3.6 Access Grafana Dashboards

Grafana starts with the Vitess cluster and provides monitoring dashboards:

- **URL:** <http://localhost:3001>
- **Username:** admin
- **Password:** admin

Pre-configured dashboards include Vitess cluster metrics, tablet health, and query performance.

3.7 Restart VTAdmin Web (if needed)

VTAdmin starts with the Vitess cluster and can be accessed at <http://localhost:14201> If it shows a blank page, rebuild:

```
cd docker/vitess/web/vtadmin
npm run build
cd ../../../../..
docker compose -f docker/vitess/docker-compose.yml restart vtadmin-web
```

4. Populating Data

4.1 Load Article Files into HDFS

The HDFS container expects sample data (text files, images, videos) at `tools/db_gen/`. This directory should contain: - `bbc_news_texts/` - Text files for article content - `image/` - JPG images for articles - `video/` - Video files (FLV format)

Load article text, images, and video files into HDFS:

```
docker exec hdfs-namenode /scripts/load-articles.sh
```

This creates `/articles/` in HDFS with per-article directories containing text, images, and videos.

Verify the data was loaded:

```
# List article directories
docker exec hdfs-namenode hdfs dfs -ls /articles | head -20

# Check a specific article
docker exec hdfs-namenode hdfs dfs -ls /articles/article0
```

4.2 Insert Users, Articles, and Reads into Vitess

```
# Quick test (50 users, 50 articles, 250 reads)
./tools/bulk_insert_helper.sh

# Standard dataset (10000 users, 10000 articles, 50000 reads)
./tools/bulk_insert_helper.sh --standard

# Custom sizes
./tools/bulk_insert_helper.sh --users 5000 --articles 5000 --reads 25000
```

4.3 Verify Data Population

```
# Connect to VTGate
mysql -h 127.0.0.1 -P 15306

# Check record counts
SELECT COUNT(*) FROM user_keyspace.user;
SELECT COUNT(*) FROM article_keyspace.article;
SELECT COUNT(*) FROM read_keyspace.read;
SELECT COUNT(*) FROM beread_keyspace.beread;
SELECT COUNT(*) FROM popularrank_keyspace.popularrank;
```

After data population: - The **Grafana dashboards** (<http://localhost:3001>) will show database metrics and query activity - The **client website** (<http://localhost:3000>) will display users, articles, and read statistics

5. Running Backup Tablets (Optional)

The backup tablets in Cell3 provide additional replicas for high availability and failover testing.

5.1 Create Backups Before Starting (Recommended)

If the primary tablets have been running for a while, the MySQL binary logs may not cover the entire history needed for new replicas to catch up. Create backups first:

```
cd docker/vitess
docker compose exec vtctld /script/backup-all-shards.sh
```

This creates backups of all shards that the new backup tablets will restore from.

5.2 Start Backup Tablets

```
cd docker/vitess
./backup-tablets.sh up
```

This starts: - Backup replicas for all keyspaces (user, article, read, beread, popularrank) - VTGate Cell3 on port 15308

5.3 Access Backup Replicas via Client

Start an additional Java server pointing to the Cell3 VTGate:

```
DB_URL="jdbc:mysql://127.0.0.1:15308/" PORT=8083 bazel run //app:server
```

The client can then access backup replicas via <http://localhost:8083>

5.4 Test Failover

With backup tablets running, you can test automatic failover by simulating a database crash:

1. Simulate a MySQL crash on a primary tablet:

```
docker compose exec vtablest_article_shard1_primary pkill -9 mysqld
```

2. Observe promotion: VTOrch will detect the failure and promote a replica to primary (typically within 10-30 seconds). Monitor in VTAdmin (<http://localhost:14201>) or check:

```
docker exec vitess-vtctld-1 vtctldclient --server localhost:15999
↵ GetTablets
```

3. Verify service continuity: Queries should continue to work through VTGate.

Note: Using `docker stop` on a tablet container is not recommended for failover testing as it causes the tablet to disappear abruptly from the topology, leaving stale records that can cause conflicts on restart. Killing `mysqld` within the container simulates a realistic database crash while keeping the tablet process running to handle recovery properly.

5.5 Stop Backup Tablets

```
cd docker/vitess
./backup-tablets.sh down
```

This stops the backup tablets and removes them from the Vitess topology.

5.6 Recover from Failed Startup

If backup tablets fail to start properly due to system resource pressure or other issues, use the `clean` command to reset to a valid state:

```
cd docker/vitess
./backup-tablets.sh clean
```

This removes all backup tablet data directories and containers. After cleaning, you can start fresh with `./backup-tablets.sh up`.

6. Running a Secondary Datacenter DC2 (Optional)

DC2 is a completely independent Vitess cluster that starts as a clone of DC1. The two datacenters are disconnected—changes made in one are not replicated to the other.

6.1 Start DC2

The `spawn` script creates backups of DC1 (since there is no binlog replication between datacenters) and starts DC2:

```
cd docker/vitess/dc2
./spawn-dc2.sh
```

This will: 1. Create backups of all DC1 shards 2. Start the DC2 cluster (tablets restore from these backups) 3. Restart VTAdmin to show both clusters

DC2 endpoints: - **VTGate Cell1:** localhost:25306 - **VTGate Cell2:** localhost:25307 - **Consul UI:** http://localhost:18500 - **VTctld:** http://localhost:25000

6.2 Validate in VTAdmin

Open VTAdmin at <http://localhost:14201>. You should now see both clusters: - **Helvetia DC1** - the original cluster - **Helvetia DC2** - the cloned cluster

6.3 Connect Client to DC2

Start additional Java servers pointing to DC2's VTGates:

```
# DC2 Cell1 server
DB_URL="jdbc:mysql://127.0.0.1:25306/" PORT=8091 bazel run //app:server &

# DC2 Cell2 server
DB_URL="jdbc:mysql://127.0.0.1:25307/" PORT=8092 bazel run //app:server &
```

The client can then access DC2 via: - <http://localhost:8091> (DC2 Cell1) - <http://localhost:8092> (DC2 Cell2)

6.4 Verify Independence

Changes made through DC2 servers will **not** be mirrored to DC1, and vice versa. You can verify this by:

1. Creating a new user via DC2 (<http://localhost:8091>)
2. Checking that the user does not appear in DC1 (<http://localhost:8081>)

This demonstrates the disconnected multi-datacenter architecture.